Project Based Learning (PBL)
Report

Python Programming

B.Tech (CSE), III Sem

HOSPITAL QUEUE MANAGEMENT SYSTEM

Submitted by

Aditya Kumar (2502220100068)
Aditi Singh (2502220100061)
Adarsh Sharma (250222010052)

Under the supervision of

Mr. Nishant Awana
Assistant Professor

Submitted to

Department of Computer Science and Engineering & Allied Branches
ITS Engineering College, Greater Noida

2026–27

I.T.S Engineering College
A unit of Durga Charitable Society
46, Knowledge Park-III, Greater Noida, Gautam Budh Nagar (U.P)-201310
Ph: +91 (0120) 2331000, Toll Free : 180018008040
E-mail: engg.gn@its.edu.in Website: www.itsengg.edu.in

Certificate

This is to certify that the Project Based Learning (PBL) report entitled “Hospital Queue Management System” has been successfully carried out by the following students of B.Tech (Computer Science & Engineering) as a part of the Project Based Learning during the academic session 2026–2027.

Submitted by:

- Name: Aditya Kumar Roll No.: 2502220100068
- Name: Aditi Singh Roll No.: 2502220100061
- Name: Adarsh Sharma Roll No.: 250222010052

The work has been completed under the guidance of **Mr. Nishant Awana**. The project is a part of continuous internal assessment under the PBL framework and reflects the students’ effort in applying theoretical knowledge to solve real-world problems.

To the best of our knowledge, this work is original and has not been submitted elsewhere for any other academic purpose.

(Signature)
Mr. Nishant Awana

Mapping of the Project to Sustainable Development Goals (SDGs)

Project Name	Problem Description	SDG No. : SDG Name
Hospital Queue Management System	Managing normal and emergency patients in an organized queue so that normal patients are served in arrival order and emergency patients are handled according to priority.	**SDG 3: Good Health and Well-being**

Justification (in point wise):

- The system supports organized patient handling in a healthcare setting.
- Normal patients are stored in a queue and served using the First In, First Out (FIFO) principle.
- Emergency patients are assigned High, Medium, or Low priority so higher-priority cases are considered first.
- The project demonstrates how a simple digital system can help organize hospital workflow.
- The project is related to SDG 3: Good Health and Well-being.

CONTENTS

- Certificate
- Mapping with SDG with Justification
- Abstract
- Problem Statement and Objectives
- Methodology/Design
- Python Data Structures and Main Modules
- System Flow
- Implementation with relevant screenshots
- Results and Discussion
- Conclusion
- References

PROJECT GITHUB REPOSITORY

https://github.com/rko443028-maker/Hospital-Queue-Management-System

ABSTRACT

The Hospital Queue Management System is a Python-based, menu-driven application designed to organize patients waiting for medical services in a hospital. The user can register patients, view waiting queues, search for a patient, and serve the next patient according to the patient type and priority.

The system maintains two waiting collections. Normal patients are stored in a deque and are served according to the First In, First Out (FIFO) principle. Emergency patients are stored in a list and sorted according to priority: High, Medium, and Low. High priority is served first. Patients with the same priority retain their registration order.

Each patient receives an automatically generated unique patient ID starting from 1001. Patient information includes ID, name, age, medical problem, and priority. The application provides options for adding normal and emergency patients, serving the next patient, displaying queues, and searching waiting patients by ID or name.

The objective is to demonstrate practical use of Python classes, lists, deque, functions, loops, conditional statements, exception handling, and searching. The project provides an educational example of applying fundamental programming concepts to a real-world hospital scenario.

PROBLEM STATEMENT AND OBJECTIVES

PROBLEM STATEMENT

Managing patients in a hospital requires an organized method for deciding the order in which patients are attended. In a simple queue, patients are served in the order of arrival. Emergency patients, however, may need attention before normal patients.

The problem is to develop a simple Hospital Queue Management System in Python that maintains normal patients in arrival order and emergency patients according to priority. The system should support patient registration, serving the next patient, viewing queues, and searching for patient information.

OBJECTIVES

1. Develop a console-based Hospital Queue Management System using Python.

2. Maintain normal patients using FIFO order.

3. Manage emergency patients with High, Medium, and Low priority.

4. Generate a unique Patient ID automatically for each registration.

5. Provide separate options for normal and emergency registration.

6. Serve waiting emergency patients before normal patients.

7. Display the normal and emergency waiting queues.

8. Search waiting patients by ID or name.

9. Handle empty queues and invalid user input.

10. Demonstrate Python classes, lists, deque, functions, and exception handling.

METHODOLOGY / DESIGN

The application is a console-based program developed in Python. A menu is shown repeatedly until the user chooses Exit. Each menu option calls a separate function, which keeps the program easy to understand and maintain.

Normal patients are stored in collections.deque. New normal patients are added at the rear and the first patient is removed from the front. Emergency patients are stored in a list and sorted by priority number: 1 for High, 2 for Medium, and 3 for Low. The system checks the emergency collection first when serving a patient.

A Patient class groups each patient's details together. A counter assigns IDs beginning at 1001. Input validation checks numeric values such as age, ID, priority, and menu choice. ValueError exceptions are handled so invalid numeric input does not crash the program.

Overall process

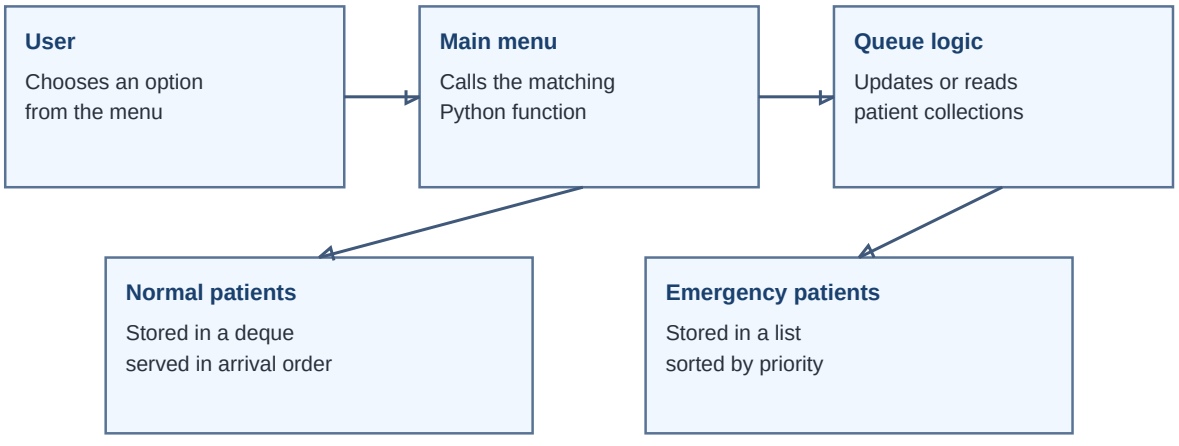
Start → Display menu → Read choice → Run selected function → Show result → Return to menu. If Exit is selected, display a closing message and stop.

Software requirements

- Python 3.x
- Any Python editor or IDE (for example, IDLE or Visual Studio Code)
- Terminal/command prompt for running the program

DESIGN AT A GLANCE

How the main parts of the Python application work together



Key idea: keep registration, storage, searching, and serving as separate tasks.

PYTHON DATA STRUCTURES USED

1. Patient class

The Patient class stores the information belonging to one patient. Its constructor receives the patient ID, name, age, problem, and priority. The display() method prints the stored details.

```
class Patient:
    def __init__(self, patient_id, name, age, problem, priority=0):
        self.patient_id = patient_id
        self.name = name
        self.age = age
        self.problem = problem
        self.priority = priority
```

2. Automatic Patient ID generation

The ID counter starts from 1001. When a patient is created, the current counter value is assigned to the patient and then increased by one.

```
next_patient_id = 1001
patient = Patient(next_patient_id, name, age, problem, priority)
next_patient_id += 1
```

This approach generates IDs automatically during one program run and avoids asking the user to type the ID.

EXAMPLE: ONE PATIENT RECORD

A Patient object groups related details into one place

Patient object

Patient ID: 1001
Name: Rahul
Age: 35
Problem: Fever
Type: Normal

Why use a class?

· Keeps fields together
· Makes display consistent
· Easy to pass between functions
· New patients use the same structure

Object-oriented idea

The class is the template; each Patient object is one individual patient's record.

The ID counter increases after each registration, so the next patient receives 1002.

NORMAL QUEUE AND EMERGENCY PRIORITY

3. Normal patient queue

A deque from Python's collections module is used for normal patients. append() inserts a patient at the rear, while popleft() removes the patient at the front. This follows FIFO order.

```
from collections import deque
normal_queue = deque()
normal_queue.append(patient)
next_patient = normal_queue.popleft()
```

4. Emergency patient collection

Emergency patients are stored in a list. The priority value is sorted in ascending order, so High (1) appears before Medium (2) and Low (3). Python's stable sorting keeps patients with equal priority in their registration order.

```
emergency_queue = []
emergency_queue.append(patient)
emergency_queue.sort(key=lambda item: item.priority)
next_patient = emergency_queue.pop(0)
```

The program always checks the emergency list before the normal queue when serving the next patient.

FIFO EXPLAINED WITH A SIMPLE EXAMPLE

FIFO means First In, First Out

Normal queue: patients leave in the same order that they joined.



FRONT · served first **REAR · newest**

Python operation

append(patient): join at rear
popleft(): remove from front

Emergency list

Priority 1 = High
Priority 2 = Medium
Priority 3 = Low

Remember: FIFO applies to normal patients; emergency patients are considered first by priority.

MAIN MODULES

1. Add Normal Patient

This module asks for the patient's name, age, and medical problem. It assigns priority 0, generates a Patient ID, and adds the patient to the normal deque.

```
patient = read_patient_details(0)
normal_queue.append(patient)
print("Normal patient registered successfully.")
```

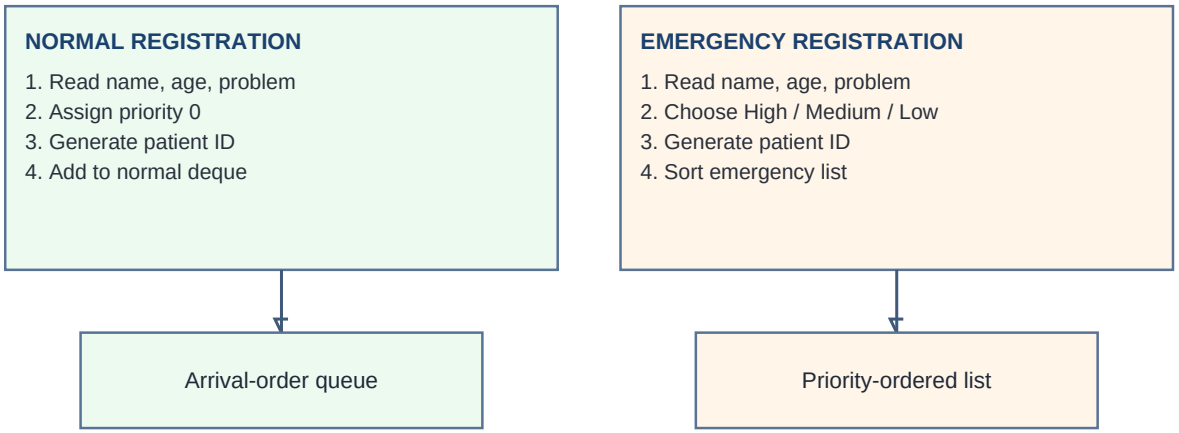
2. Add Emergency Patient

The user selects High, Medium, or Low priority. The selected priority is stored in the Patient object and the emergency list is sorted so that the highest-priority patient is served first.

```
patient = read_patient_details(priority)
emergency_queue.append(patient)
emergency_queue.sort(key=lambda item: item.priority)
```

REGISTRATION: TWO PATHS

The program collects the same basic details but stores patients differently



Both paths finish by showing a confirmation and the newly assigned Patient ID.

SERVING AND VIEWING PATIENTS

3. Serve Next Patient

The system first checks whether an emergency patient is waiting. If yes, the first item in the sorted emergency list is removed. Otherwise, it serves the patient at the front of the normal queue. If both are empty, an informative message is displayed.

```
if emergency_queue:
    patient = emergency_queue.pop(0)
elif normal_queue:
    patient = normal_queue.popleft()
else:
    print("No patients are waiting.")
```

4. View Patient Queues

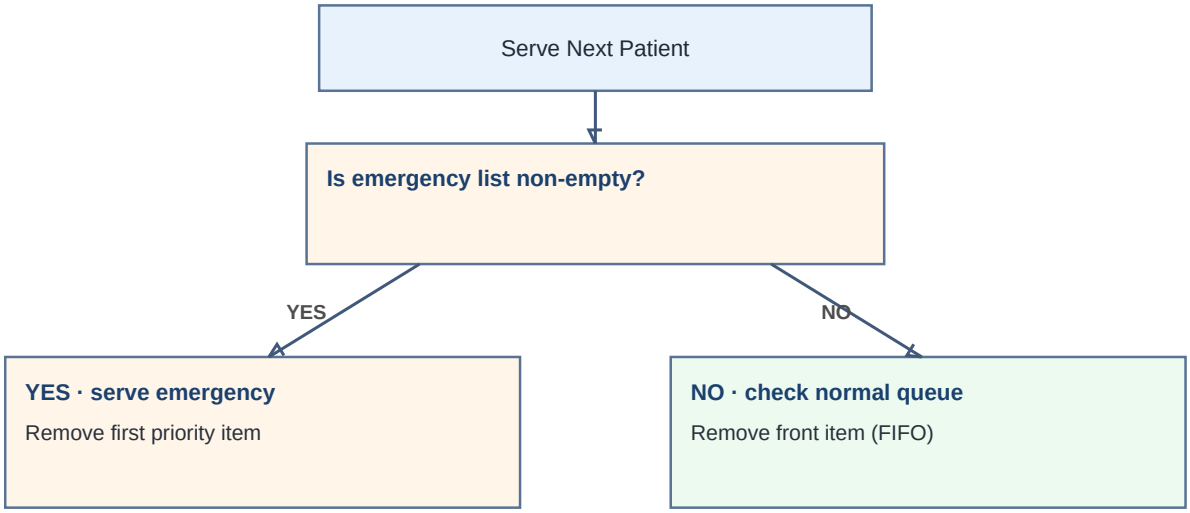
The normal and emergency queue options display patient details without removing patients. A loop reads each item in the selected collection. If a collection is empty, the program prints an empty-queue message.

```
for patient in normal_queue:
    patient.display()

for patient in emergency_queue:
    patient.display()
```

SERVE NEXT PATIENT: DECISION FLOW

Emergency patients are checked before the normal queue



If both collections are empty, the program prints ·No patients are waiting.·

SEARCH PATIENT AND INPUT VALIDATION

5. Search Patient

The search module offers search by Patient ID or Patient Name. It checks the patients still waiting in both collections. Name search uses casefold() so uppercase and lowercase letters are treated alike. Patients already served are no longer present in the waiting queues and cannot be found by this search.

```
all_patients = list(normal_queue) + list(emergency_queue)
matches = [p for p in all_patients
           if p.patient_id == target]
```

6. Input Validation

The program checks that age is a positive integer, priority is between 1 and 3, and menu choice is one of the available options. try/except blocks handle non-numeric input without terminating the program.

```
try:
    age = int(input("Enter patient age: "))
except ValueError:
    print("Please enter age as a number.")
```

SEARCH AND VALIDATION

These checks help the program respond safely to user input

Search patient

Combine waiting collections
Compare ID or name
Display match / not found

Validate input

Age must be a positive number
Priority must be 1, 2, or 3
Invalid menu input is rejected

Important limitation

Search checks patients still waiting. Once served, a patient is removed from the queues and is not found in this version.

Using try/except around int(...) prevents non-numeric input from crashing the program.

SYSTEM FLOW

The system follows a menu-driven flow. On startup, the main menu is displayed. The user chooses an operation such as registering a normal patient, registering an emergency patient, serving the next patient, viewing a queue, searching, or exiting.

When a patient is registered, a unique ID is assigned and the patient is stored in the correct collection. Normal patients remain in arrival order. Emergency patients are ordered by priority. During service, the system checks the emergency collection first; if it is empty, the first normal patient is served.

After each operation, control returns to the menu. The program ends only when the user chooses Exit.

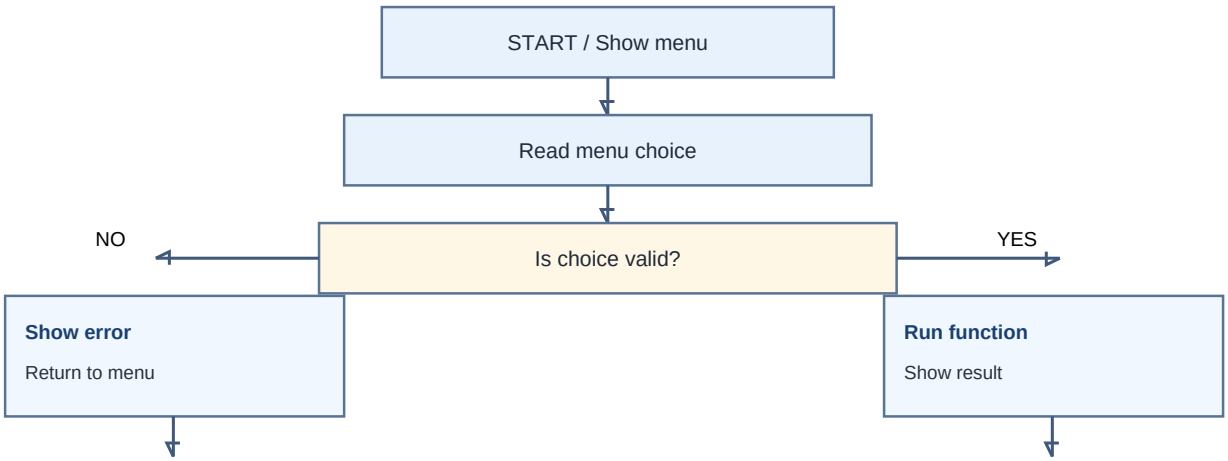
System flowchart

START
Display main menu
Read user choice
Valid option?
Run selected operation
Return to menu / Exit

Flowchart sequence is represented above in a simple printable format. The “Valid option?” decision leads to the selected function for a valid option or an error message for an invalid choice.

DETAILED SERVICE FLOW

A readable flow from the menu to the next operation



Both paths return to the menu unless the user selects Exit.

IMPLEMENTATION WITH RELEVANT CODE

The project is implemented in Python using a modular approach. The Patient class represents a patient, while separate functions handle registration, serving, queue display, searching, and the main menu.

The code is kept in a single file named hospital_queue_management.py. It can be run with Python 3 by opening a terminal in the file's folder and entering:

```
python hospital_queue_management.py
```

The following pages show the important parts of the implementation. The complete runnable source file is also provided separately so that it can be uploaded to the GitHub repository without missing code.

Patient class

```
class Patient:
    def __init__(self, patient_id, name, age, problem, priority=0):
        self.patient_id = patient_id
        self.name = name
        self.age = age
        self.problem = problem
        self.priority = priority
```

HOW TO RUN THE PROJECT

Basic execution steps for a student using a terminal

1. Save file

Keep hospital_queue_management.py in a known folder.

2. Open terminal

Open Command Prompt or the terminal in your editor.

3. Run command

Type: python hospital_queue_management.py

4. Use menu

Enter an option from 1 to 7 and follow prompts.

If 'python' is not recognized, check that Python 3 is installed and added to PATH.

IMPLEMENTATION: PATIENT DETAILS AND IDs

The display() method prints patient details in a readable format. A priority dictionary converts numeric priority into the labels Normal, High, Medium, and Low.

```
def display(self):
    level = {0: "Normal", 1: "High",
             2: "Medium", 3: "Low"}
    print(f"ID: {self.patient_id} | Name: {self.name}")
    print(f"Age: {self.age} | Problem: {self.problem}")
    print(f"Type: {level[self.priority]}")
```

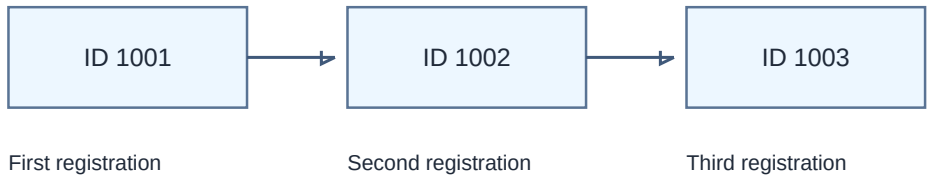
Automatic ID generation uses a shared counter. Each call to the registration function assigns the current value and increments it for the next patient.

```
next_patient_id = 1001
patient = Patient(next_patient_id, name, age, problem, priority)
next_patient_id += 1
```

This ID counter remains unique during the current run of the program. Persistent IDs across separate runs would require a database or file storage, which is outside the current version.

PATIENT ID EXAMPLE

IDs are assigned automatically during one program run



Note about storage

The counter restarts at 1001 when the program is launched again. A database or file would be needed to preserve IDs between runs.

IMPLEMENTATION: NORMAL PATIENT REGISTRATION

The normal registration function collects details through read_patient_details(). It passes priority 0, then adds the resulting Patient object to the deque.

```
def add_normal_patient():
    patient = read_patient_details(0)
    normal_queue.append(patient)
    print("Normal patient registered successfully.")
    patient.display()
```

The read_patient_details() function checks that the name is not blank and age is a positive integer. The patient ID is assigned automatically. This makes registration easier and reduces basic input errors.

Example interaction

```
Enter patient name: Rahul
Enter patient age: 35
Enter medical problem: Fever
Normal patient registered successfully.
ID: 1001 | Name: Rahul | Age: 35
```

NORMAL REGISTRATION: EXAMPLE

Illustrative input and the expected confirmation

Sample console

Enter patient name: Rahul
Enter patient age: 35
Enter medical problem: Fever
Normal patient registered successfully.
ID: 1001 | Name: Rahul | Age: 35

What happens behind the scenes?

The details are used to create a Patient object. The object is appended to normal_queue.

It joins at the rear and remains waiting until served or the program closes.

IMPLEMENTATION: EMERGENCY REGISTRATION

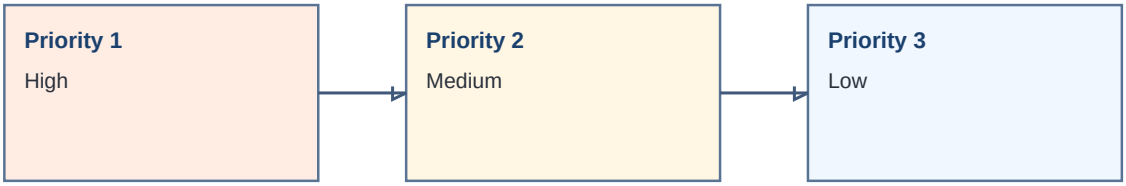
Emergency registration first asks the user to select a priority. Only 1, 2, or 3 is accepted. The selected priority is passed to the patient-detail function and the new patient is placed in priority order.

```
def add_emergency_patient():
    print("1. High  2. Medium  3. Low")
    while True:
        try:
            priority = int(input("Choose priority (1-3): "))
            if priority in (1, 2, 3):
                break
            print("Enter 1, 2, or 3.")
        except ValueError:
            print("Please enter a number.")
```

After collecting details, the patient is inserted into the emergency list and the list is sorted by priority. High-priority patients therefore appear before medium- and low-priority patients.

EMERGENCY PRIORITY EXAMPLE

A smaller priority number means more urgent in this project



Example

If Low is registered first but High is registered later, sorting places High first for service.

Priority rules are part of the program demonstration; actual medical triage must be performed by qualified staff.

IMPLEMENTATION: SERVE NEXT PATIENT

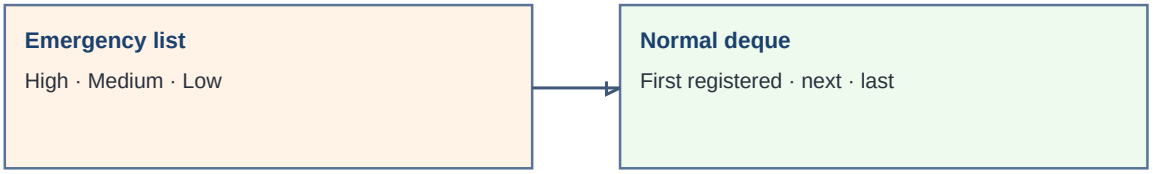
This function follows the project's main service rule: emergency patients are considered before normal patients. The patient removed from the relevant collection is displayed on the screen.

```
def serve_next_patient():
    if emergency_queue:
        patient = emergency_queue.pop(0)
        print("Serving emergency patient:")
    elif normal_queue:
        patient = normal_queue.popleft()
        print("Serving normal patient:")
    else:
        print("No patients are waiting.")
        return
    patient.display()
```

If both collections are empty, the function prints a message and returns safely. It does not attempt to remove an item from an empty collection.

SERVING PATIENTS: EXAMPLE

The system chooses the next patient using the following order



Service rule

If any emergency patient is waiting, serve the first priority item.
Otherwise, serve the front of the normal queue.

When both queues are empty, the function returns safely without removing an item.

IMPLEMENTATION: NORMAL PATIENT QUEUE

The normal queue view displays the patients in arrival order. The function checks for an empty queue first, then loops through the deque and displays each patient.

```
def show_normal_queue():
    if not normal_queue:
        print("Normal queue is empty.")
        return
    print("NORMAL PATIENT QUEUE")
    for patient in normal_queue:
        patient.display()
        print("-" * 35)
```

The display operation does not remove any patient. The deque remains unchanged and the same patient can later be served by the Serve Next Patient option.

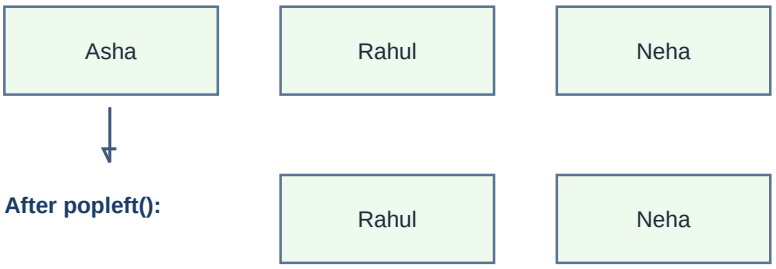
Expected console result

```
NORMAL PATIENT QUEUE
ID: 1001 | Name: Rahul | Age: 35
Problem: Fever | Type: Normal
-----------------------------------
```

FIFO QUEUE: BEFORE AND AFTER

Viewing the queue does not remove patients; serving does

Before serving:



After popleft():

Asha has been served; Rahul is now at the front.

Why deque?

Adding at the rear and removing at the front are natural queue operations.

IMPLEMENTATION: EMERGENCY PATIENT QUEUE

The emergency queue view lists patients according to priority order. It does not remove patients from the waiting list.

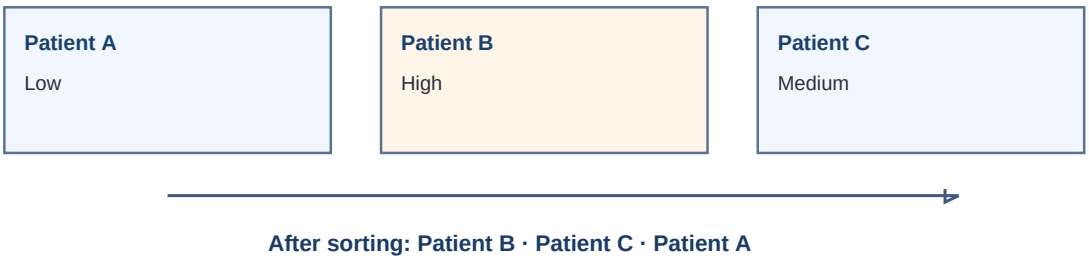
```
def show_emergency_queue():
    if not emergency_queue:
        print("Emergency queue is empty.")
        return
    print("EMERGENCY PATIENT QUEUE")
    for patient in emergency_queue:
        patient.display()
        print("-" * 35)
```

For example, if a Low-priority patient is registered before a High-priority patient, the High-priority patient will still appear first after sorting. Patients with equal priority retain their arrival order.

EMERGENCY QUEUE: PRIORITY ORDER

The displayed order is based on priority, not only arrival time

Suppose patients arrive in this order:



Stable ordering

Patients with the same priority keep their registration order because Python's sort is stable.

IMPLEMENTATION: SEARCH BY PATIENT ID

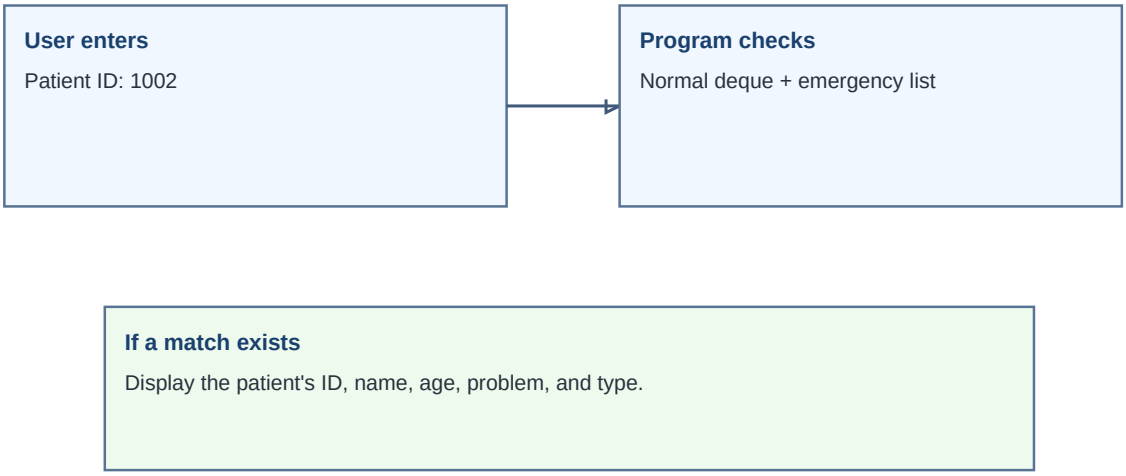
Search by ID converts the user's entry to an integer. It then checks the normal and emergency waiting collections. If the ID matches, the patient details are displayed; otherwise a not-found message is shown.

```
all_patients = list(normal_queue) + list(emergency_queue)
try:
    target = int(input("Enter patient ID: "))
except ValueError:
    print("Patient ID must be a number.")
    return
matches = [p for p in all_patients
           if p.patient_id == target]
```

The search checks patients who are still waiting. Since served patients are removed from their queues, this version does not keep a permanent history of served patients.

SEARCH BY ID: EXAMPLE

Search only checks patients who are currently waiting



If the patient has already been served, the ID will not be found in the waiting collections.

IMPLEMENTATION: SEARCH BY PATIENT NAME

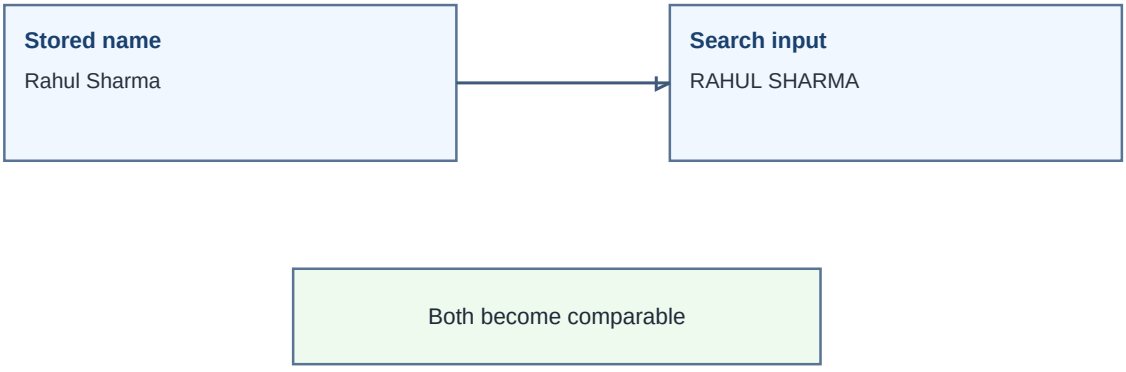
The user can also search by name. The entered name and stored names are compared using casefold(), allowing case-insensitive matching.

```
target = input("Enter patient name: ").strip().casefold()
all_patients = list(normal_queue) + list(emergency_queue)
matches = [p for p in all_patients
           if p.name.casefold() == target]
```

If one or more matching patients are found, their details are displayed. If there is no match, the program informs the user. The search does not change the waiting queues.

SEARCH BY NAME: EXAMPLE

casefold() makes name matching ignore letter case



The search does not change the queue. It only reads records and reports matching details.

IMPLEMENTATION: MAIN MENU

The main() function repeatedly displays the menu and calls the required module. The loop continues until the user selects option 7.

```
while True:
    print("1. Add normal patient")
    print("2. Add emergency patient")
    print("3. Serve next patient")
    print("4. View normal queue")
    print("5. View emergency queue")
    print("6. Search waiting patient")
    print("7. Exit")
    choice = input("Enter your choice (1-7): ").strip()
```

The choice is compared with the available menu options. Any other entry produces an invalid-choice message. The program starts through the standard Python main guard:

```
if __name__ == "__main__":
    main()
```

MAIN MENU: OPTIONS AT A GLANCE

The menu repeats after each operation until Exit is selected

1. Add normal patient

2. Add emergency patient

3. Serve next patient

4. View normal queue

5. View emergency queue

6. Search waiting patient

7. Exit

A separate function handles each task, making the code easier to read, test, and maintain.

RESULTS AND DISCUSSION

The program can be tested using normal registration, emergency registration, serving, queue display, search, invalid input, and exit operations. The table below lists the expected results for common test cases.

S. No.	Test case	Expected result	Status
1	Add normal patient	Patient is registered and added to normal queue	Pass
2	Add emergency patient	Patient is stored with selected priority	Pass
3	Invalid priority	User is asked to enter 1, 2, or 3	Pass
4	Serve next patient	Emergency first; otherwise FIFO normal patient	Pass
5	View normal queue	Waiting normal patients are displayed	Pass
6	View emergency queue	Emergency patients are displayed by priority	Pass
7	Search by ID	Matching waiting patient is displayed	Pass
8	Search by name	Matching waiting patient is displayed	Pass
9	Unknown ID/name	Not-found message is displayed	Pass
10	Empty queue	Empty-queue message is displayed	Pass
11	Invalid age input	Program requests a valid positive number	Pass
12	Invalid menu choice	Invalid-choice message is displayed	Pass
13	Exit	Program terminates normally	Pass

DISCUSSION AND OVERALL RESULT

The design separates normal and emergency patients so that each group follows its own service rule. The deque is suitable for normal patients because patients are appended at the rear and removed from the front. The emergency list is sorted by priority before service.

Automatic Patient ID generation works during a single run. Input checks reduce errors caused by invalid age, priority, and menu entries. Queue display and search operations read the stored information without removing waiting patients.

The application is a learning prototype and does not replace a hospital's real patient record system. Data is held in memory, so it is cleared when the program closes. A future version could add persistent storage, a graphical interface, authentication, and a database.

Overall result

The main functions are implemented in the accompanying Python source file. Run the file and perform the listed test cases to verify the results on the target computer.

CONCLUSION

The Hospital Queue Management System demonstrates how Python programming concepts can be used to organize a simple patient queue. Normal patients are served according to FIFO order, while emergency patients are served according to High, Medium, and Low priority. The project includes automatic IDs, patient registration, queue display, searching, and input validation.

REFERENCES

1. Python Software Foundation, Python 3 documentation: https://docs.python.org/3/

2. Python collections.deque documentation: https://docs.python.org/3/library/collections.html#collections.deque

3. Python built-in data types and list sorting documentation: https://docs.python.org/3/tutorial/datastructures.html

4. Python exception handling tutorial: https://docs.python.org/3/tutorial/errors.html

5. PBL report format provided by the Department of Computer Science and Engineering & Allied Branches, ITS Engineering College, Greater Noida, Academic Session 2026–2027.

6. Python programming learning resources used for understanding classes, functions, lists, loops, and exception handling.

GITHUB REPOSITORY

https://github.com/rko443028-maker/Hospital-Queue-Management-System

— End of Report —